



Gradients and Optimization

Deep Neural Networks
Session 09
Pramod Sharma
pramod.sharma@prasami.com

2 Agenda

- Loss / Cost Optimization
- Stochastic Gradient Descent (SGD) and others
- Momentum Learning Rates
- Adaptive Learning Rates

5/8/2026

pra-sami

3

Optimization in Neural Networks

- Training a neural network is formulated as an optimization problem
- Objective: minimize a loss (cost) function by adjusting model parameters
- Learning Tasks
 - ❖ Classification: Predict probability distribution over classes
 - ❖ Regression: Predict continuous numerical values
- Model Perspective
 - ❖ Model learns parameters (weights W and biases b)
 - ❖ Goal: map inputs $\rightarrow$ outputs to approximate target values

5/8/2026

pra-sami

4

Optimization Problem

- Loss Function
 - ❖ Measures prediction error: $\ell(a, y)$
 - ❖ where:
 - Model Output $a = f(x; W; b)$
 - Ground truth y
- Overall cost: $J(W, b) = \frac{1}{m} \sum \ell(a, y)$
- Gradient (Error Signal)
 - ❖ Gradient = rate of change of loss w.r.t parameters
 - ❖ Indicates direction to adjust parameters to reduce error
 - ❖ Compute gradient $\frac{\partial J}{\partial W} \cdot \frac{\partial J}{\partial b}$
 - ❖ Update weights $W = W - \alpha \cdot \frac{\partial J}{\partial W} \cdot X$
 - ❖ Similarly $b = b - \alpha \cdot \frac{\partial J}{\partial b}$ Where α is defined as learning rate (step size)

- ❖ Most learnable capacity lies in weights across layers
- ❖ Optimization iteratively reduces loss $\rightarrow$ improves predictions

5/8/2026

pra-sami

5

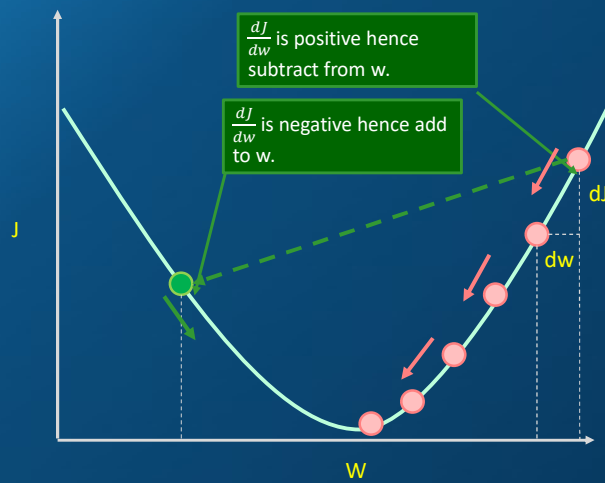
Loss / Cost Optimization

5/8/2026

pra-sami

6

Optimization Problem

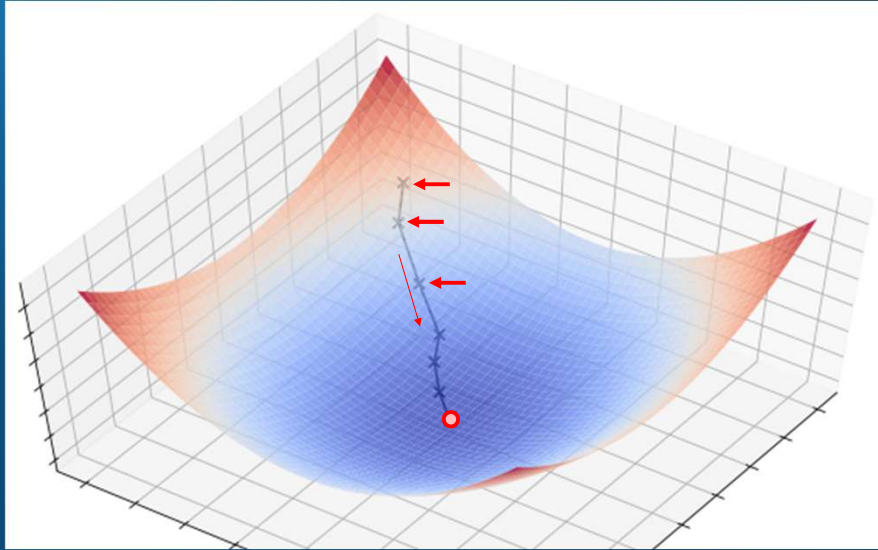


5/8/2026

pra-sami

7

Loss / Cost Optimization



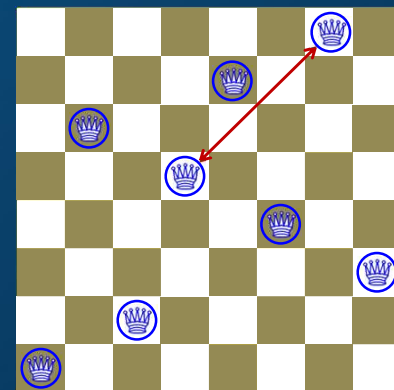
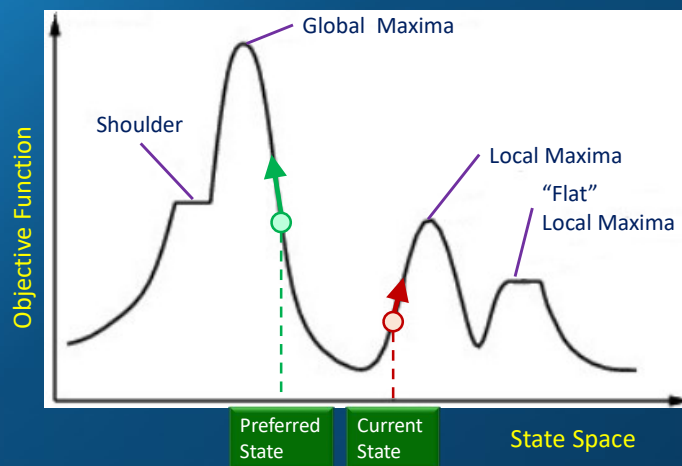
5/8/2026

pra-sami

8

State Space Landscape

- Problem: depending on initial state, can get stuck in local maxima/minima

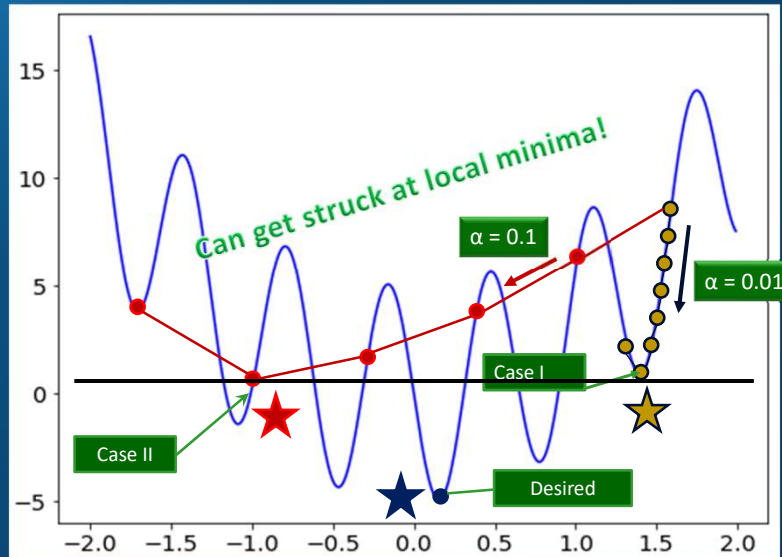
Local Minimum with $h = 1$

5/8/2026

pra-sami

9

Learning Rate : Difficult to assess what's doing to work?



5/8/2026

pra-sami

10

Gradient – Tough Terrain



5/8/2026

pra-sami

11

Learning Rate: Navigating a Tough Optimization Landscape

- ❑ Choosing the right learning rate (α) is critical for effective training
- ❑ Optimization landscape is often non-convex (multiple minima, saddle points)
- ❑ How to select the right learning rate?
- ❑ If Learning Rate is Too High
 - ❖ May overshoot the minimum
 - ❖ Causes oscillations or divergence
 - ❖ Training may fail to converge
- ❑ If Learning Rate is Too Low
 - ❖ Slow convergence (very long training time)
 - ❖ Can get stuck in plateaus or saddle points
 - ❖ Inefficient learning

Desired Behavior

- ❖ Stable and smooth convergence toward minimum
- ❖ Efficient progress without oscillations

5/8/2026

pra-samī

12

How to Select the Right Learning Rate?

Why "One Size Fits All" Fails in Deep Learning

- ❑ The Dilemma:
 - ❖ Too large = Overshoot/Diverge.
 - ❖ Too small = Slow/Stuck in plateau.
- ❑ The Solution:
 - ❖ A learning rate that adapts to the terrain (geometry of the loss landscape).
- ❑ The Analogy:
 - ❖ Fixed LR: Hiking blindfolded with a fixed step size (stumble on rocks, miss the valley).
 - ❖ Adaptive LR: Feeling the slope with your feet; take big steps on flat plains, tiny steps near the bottom.

5/8/2026

pra-samī

13

From Vanilla to Adaptive: The Evolution

- SGD (Classic) → Problem: Sensitive to LR, slow on ravines.
- SGD + Momentum → Fix: Accelerates in consistent directions (SGD+M).
- Adaptive Family (Per-parameter LR):
 - ❖ Adagrad → Idea: Adapts based on frequency (good for sparse data).
 - ❖ RMSProp → Fix: Solves Adagrad's "dying LR" problem.
 - ❖ Adam (King of the Hill) → Idea: RMSProp + Momentum.

Adaptive methods automatically tune the LR for each weight individually

5/8/2026

pra-sami

14

Gradient Descent

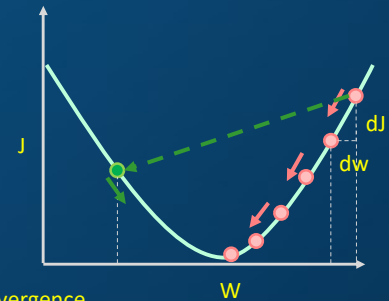
5/8/2026

pra-sami

15

Gradient Descent

- ❑ The most popular algorithm to optimize neural networks.
- ❑ What are we optimizing?
 - ❖ Cost function $J(W, b)$ — measures how wrong our predictions are.
 - For a set of X, y
- ❑ How does it work?
 - ❖ Compute gradient (slope) → Move opposite direction → Repeat until convergence.
- ❑ The Critical Hyperparameter:
 - ❖ Learning rate (α) = Step size
- ❑ The Problem:
 - ❖ Most libraries treat optimizers as black boxes.
 - Result: Students don't know why Adam works or SGD fails.



5/8/2026

pra-sâmi

16

Stochastic Gradient Descent (SGD) — The Workhorse

- ❑ What does "Stochastic" mean?
 - ❖ A randomly determined process — from Greek 'stochastikos' (skilful at aiming)
- ❑ How it works (pure form):
 - ❖ Update parameters after every single training example (1 sample at a time)

Equation:

$$W \leftarrow W - \alpha \times \frac{\partial J(W, b, x_i, y_i)}{\partial W}$$

Pros:

- ❖ Frequent updates → Fast initial progress
- ❖ Noisy gradient → Can jump out of local minima
- ❖ Works online (data arrives one sample at a time)

Cons:

- ❖ Extreme fluctuations → Loss bounces wildly
- ❖ Never converges perfectly (just wanders near minimum)
- ❖ Wastes GPU parallelization

- ❖ Our very first model.
- ❖ Pure SGD is rarely used in deep learning. Too slow, too noisy.

5/8/2026

pra-sâmi

17

Batch Gradient Descent — The Opposite Extreme

❑ What is Batch Gradient Descent?

- ❖ Update parameters once after computing gradient on the entire training dataset

❑ Equation: $\frac{\partial J}{\partial w_j} = \frac{1}{m} * \left(\sum_{i=1}^N \frac{\partial \ell(w_j, x_i, y_i)}{\partial w_j} \right)$ and $W_j = W_j - \alpha * \frac{\partial J}{\partial w_j}$

❑ Most deep learning libraries provide automatic differentiation that efficiently computes the gradient

❑ For non-convex surfaces, it converges to local minima

Epoch

Dictionary : “A long period of time, especially one in which there are new developments and great change”

ML: One cycle through the entire training dataset

5/8/2026

pra-sâmi

18

Batch Gradient Descent

What works well

Stable convergence	True gradient → smooth, predictable loss decrease
Theoretically sound	Guaranteed to find minimum for convex functions
No batch size tuning	One less hyperparameter

The deal Breaker

Computationally crushing	One gradient pass through ALL data per update
Impractical for large datasets	ImageNet (1.2M images) → Wait hours for ONE update
Cannot escape local minima	Zero noise = zero chance to jump out of bad valleys
Poor generalization	Converges to sharp minima → overfits
No online learning	New data requires full retraining
Memory bottleneck	Can't fit huge datasets in GPU memory
Hard to parallelize efficiently	Sync overhead kills speed gains

5/8/2026

pra-sâmi

19

Mini-batch Gradient Descent — What You Actually Use



□ Create small batches of fixed size → Update after each batch

□ The "Power of 2" Rule

✦ Always use batch sizes as powers of 2:

➢ 16, 32, 64, 128, 256, 512

□ Why? GPU memory architecture is optimized for these sizes.

5/8/2026

pra-sami

20

Batch vs. Mini-batch

Index	X1	X2	y
0	0.871	0.64	0
1	0.987	0.633	0
2	0.52	0.405	0
...	...	...	...
127	0.857	0.44	0
128	0.154	0.161	0
129	0.642	0.722	0
...	...	...	...
256	0.825	0.844	1
257	0.763	0.244	1
258	0.562	0.225	0
...	...	...	...
383	0.22	0.573	0
384	0.953	0.797	0
385	0.118	0.62	1
...	...	...	...
1152	0.695	0.215	0

Split entire data in mini batches of 128 rows.

Batch	Index	X1	X2	y
Batch 1	0	0.660	0.775	1
	1	0.343	0.605	1
	2	0.771	0.958	0
	...	...	...	...
	127	0.291	0.231	0
Batch 2	128	0.886	0.255	1
	129	0.630	0.873	1
	...	...	...	...
	256	0.260	0.880	0
Batch 3	257	0.002	0.263	1
	258	0.055	0.351	1
	...	...	...	...
	383	0.444	0.986	1
...	384	0.997	0.020	0
	385	0.807	0.789	0
	...	...	...	...
	...	...	...	...
	1152	0.695	0.215	0

5/8/2026

pra-sami

21

Mini-Batch Gradient Descent

Pro	Why It Matters	Con	Why It Matters	Mitigation
Frequent model updates	Learn faster than Batch GD	Batch size is a new hyperparameter	Need to tune it	Start with 16 or 32; Even less for laptop
Maintains batched computation	GPU parallelization works efficiently	Trade-off: small vs large batch	Small = noisy, Large = slow	Experiment
Fits in GPU memory	32 samples $\times$ model size = manageable	Too large $\rightarrow$ poor generalization	Large batches find sharp minima	Stay ≤ 128 for vision tasks
Enough noise to escape local minima	Randomness helps generalization	Too small $\rightarrow$ wastes GPU	Doesn't utilize parallel hardware	Use power of 2 (32, 64, 128)
Stable gradient estimates	Less noisy than pure SGD			

5/8/2026

pra-sami

22

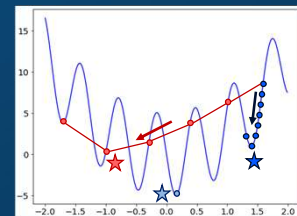
Overall Challenges — Why Static LR Fails

❑ The Learning Rate Dilemma

- ❖ What's the proper learning rate?

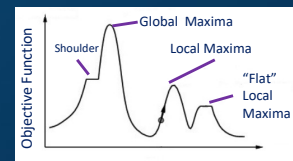
❑ LR Schedules Are Blind

- ❖ What people try first:
 - Reduce learning rate by half every 10 epochs, or use cosine decay.
- ❖ The problem:
 - Schedules are pre-defined — they don't adapt to the actual terrain.
- ❖ Example of failure:
 - You reach a plateau at epoch 5, but the schedule only reduces LR at epoch 10. You waste 5 epochs doing nothing.



❑ Local Minima vs Saddle Points (The Plateau Problem)

- ❖ Common misconception:
 - "We're afraid of getting stuck in local minima."
- ❖ The truth (for high dimensions):
 - In large neural networks, saddle points and plateaus are much more common than bad local minima.



5/8/2026

pra-sami

23

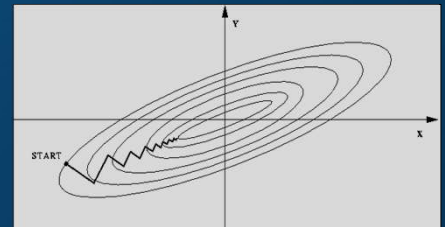
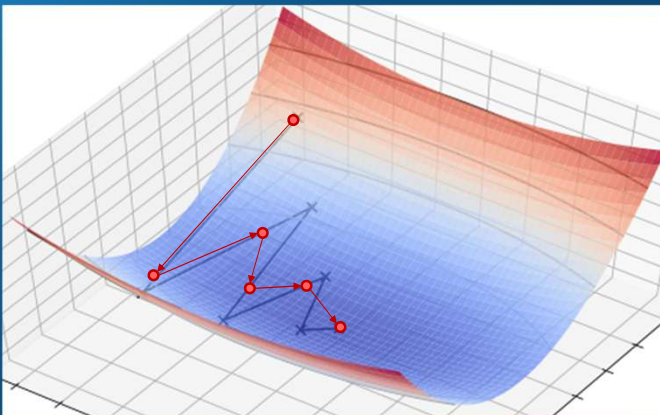
Momentum Learning Rates

5/8/2026

pra-sami

24

Momentum Gradient Descent



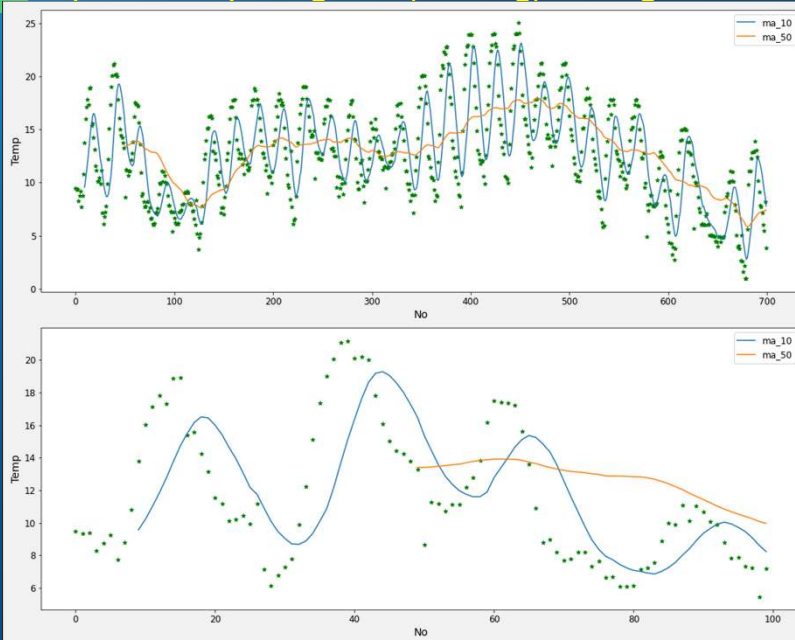
Approximate weighted averages???

5/8/2026

pra-sami

25

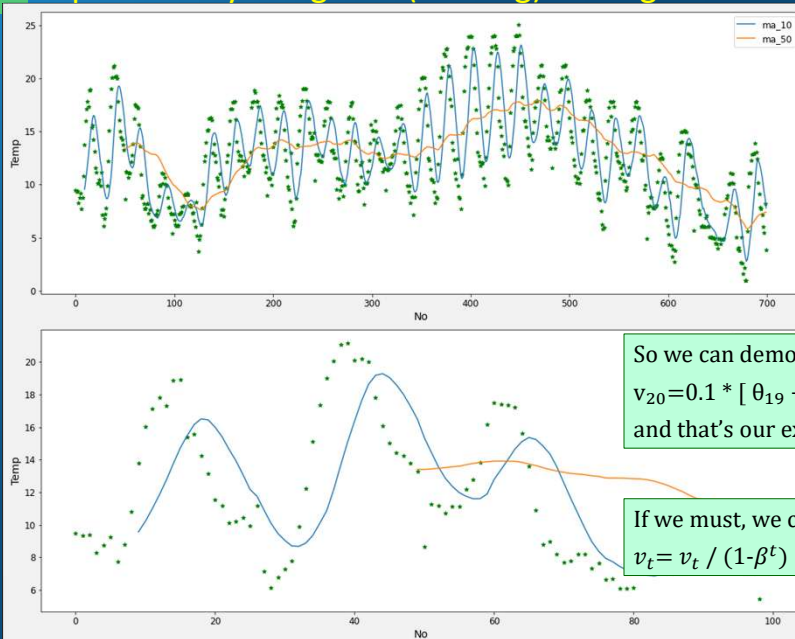
Exponentially Weighted (Moving) Average



pra-sami

26

Exponentially Weighted (Moving) Average



$$v_t = \beta v_{t-1} + (1+\beta) * \theta_{t-1}$$

So we can demonstrate that

$$v_{20} = 0.1 * [\theta_{19} + 0.9 * \theta_{19} + 0.9^2 \theta_{18} + 0.9^3 \theta_{17} + \dots]$$

and that's our exponentially weighted moving average....

If we must, we can correct the bias using following value

$$v_t = v_t / (1 - \beta^t)$$

pra-sami

27

Gradient Descent with Momentum

Momentum — "Rolling Downhill Like a Heavy Ball"

❑ Ravines in Loss Landscapes

- ❖ What is a ravine?
 - A surface that curves much more steeply in one direction than the other
- ❖ Example: In a neural network, some weights are more important than others



pra-sami

28

Gradient Descent with Momentum

Momentum — "Rolling Downhill Like a Heavy Ball"

❑ Momentum

- ❖ SGD = Taking a step, stopping, taking another step (no memory)
- ❖ Momentum = Rolling a heavy ball downhill. It builds speed, ignores small bumps, and smooths out oscillations.

❑ Momentum Gradient Descent updates as follows:

- ❖ $V_t = \beta * V_{t-1} + (1 - \beta) * \frac{\partial J}{\partial W}$
- ❖ and $W_t = W_{t-1} - \alpha * V_t$

Original Polyak "Heavy Ball" Method (1964) :

$$V_t = \beta * V_{t-1} + \alpha * \frac{\partial J}{\partial W}$$

$$W_t = W_{t-1} - V_t$$

❑ Congratulations! β is another parameter you can tune!

Some libraries are using original equation in out-of-the box implementation

5/8/2026

pra-sami

29

Momentum Gradient Descent — How It Works

- $\beta = 0.9$ (default)
- Key behaviors:
 - ❖ Velocity accumulates past gradients (memory)
 - ❖ Momentum grows over time — like rolling downhill
 - ❖ Same direction $\rightarrow$ increases velocity $\rightarrow$ faster convergence
 - ❖ Changing directions $\rightarrow$ reduces updates $\rightarrow$ dampens oscillations
- Result: Faster convergence + reduced oscillation

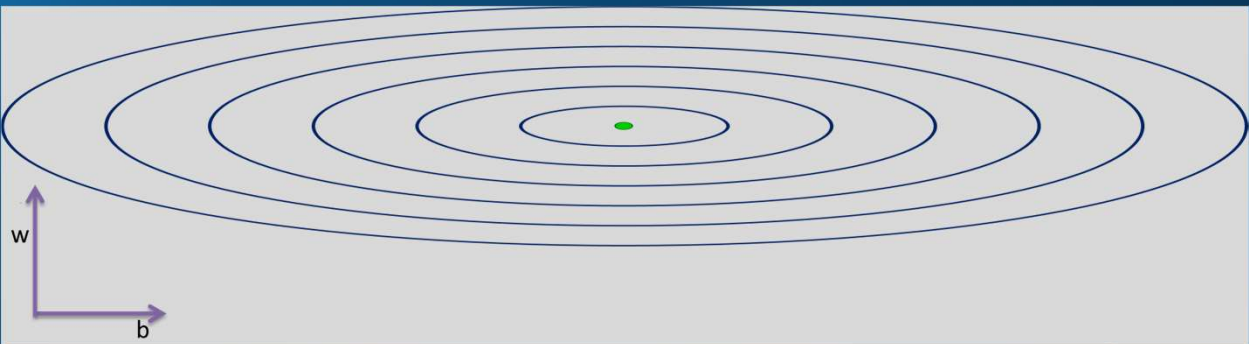
5/8/2026

pra-sami

30

Gradient Descent Example

- Compute dW and db
- Compute V_{dw} and V_{db}
- Update $V_{dw} = \beta \times V_{dw} + (1 - \beta) \times dw$ and $V_{db} = \beta \times V_{db} + (1 - \beta) \times db$
- Update $W = W - \alpha \times V_{dw}$ and $b = b - \alpha \times V_{db}$



31

Pros and Cons of Momentum

Pros

- ❑ Faster convergence
- ❑ Smooths oscillations
- ❑ Escapes shallow local minima
- ❑ Pushes through plateaus

Cons

- ❑ One more hyperparameter (β)
- ❑ Can overshoot minima
- ❑ Needs careful LR tuning
- ❑ $\beta=0.9$ not always optimal

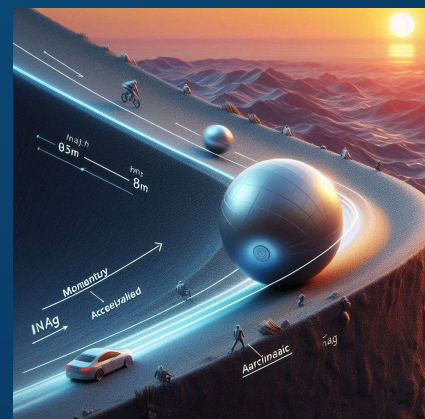
5/8/2026

pra-sami

32

Nesterov Accelerated Gradient (NAG) – A Smarter Momentum

- ❑ Nesterov Accelerated Gradient method was published by Yurii Nesterov in 1983
- ❑ Gradient Descent (GD):
 - ❖ Moves in the direction of steepest descent.
- ❑ Momentum:
 - ❖ Accumulates past gradients to speed up convergence.
- ❑ Challenges:
 - ❖ Momentum can overshoot minima due to inertia.
 - ❖ At the point of update, momentum relies on the current gradient, which may not be optimal.



5/8/2026

pra-sami

33

NAG – Look Before You Leap

- ❑ Where will I be after my momentum carries me? What's the slope there?
- ❑ First makes a partial momentum-based update, then computes the gradient.

The Three-Step Process:

- ❑ Step 1: Look ahead
 - ❖ Calculate where the current velocity would take you
 - ❖ $W_{lookahead} = W - \alpha \times V_t$
- ❑ Step 2: Compute gradient at lookahead position
 - ❖ Gradient $\nabla = \frac{\partial J}{\partial W_{lookahead}}$
- ❑ Step 3: Update velocity and weights
 - ❖ $V_t = \beta \times V_{t-1} + (1 - \beta) \times \nabla$
 - ❖ and $W = W - \alpha \times V_t$

- ❖ Momentum: Follows gradient at current position.
- ❖ NAG: Follows gradient at "lookahead" position.

5/8/2026

pra-sâmi

34

NAG vs Momentum — When to Use Which?

Aspect	Momentum	NAG
Gradient location	Current position	Lookahead position
Overshoot tendency	High (inertia un-checked)	Lower (corrected before overshoot)
Convergence speed	Fast	Faster (especially near minima)
Ravine handling	Good	Better
Saddle point escape	Moderate	Better
Computational cost	Low	Slightly higher (extra lookahead step)

5/8/2026

pra-sâmi

35

Advantages

Advantage	Why it matters
Faster convergence near minima	Corrects velocity before overshooting
Better handling of ravines	Lookahead anticipates terrain changes
Better saddle point escape	Corrective gradient prevents getting stuck
Used in modern optimizers	Some Adam variants support Nesterov momentum

5/8/2026

pra-sâmi

36

Limitations

Limitation	Explanation
Still requires tuning	Learning rate α and momentum β need tuning
Not always better	Adam or RMSProp may outperform in some scenarios
Slightly more compute	Extra lookahead step (negligible in practice)
Can still overshoot	Reduced, not eliminated

5/8/2026

pra-sâmi

37

When to Use NAG?

Scenario	Recommendation
Very steep ravines	Use NAG over standard momentum
Near minimum convergence issues	NAG helps prevent oscillation
Limited compute budget	Extra step is tiny — still worth it
Baseline comparison	Start with standard momentum (simpler)
When using Adaptive Techniques	Some implementations support Nesterov variant

5/8/2026

pra-sami

38

Adaptive Learning Rates

5/8/2026

pra-sami

39

AdaGrad (Adaptive Gradient Algorithm):

❑ SGD needs:

- ❖ Starting point to be selected carefully
- ❖ Constant learning rate
- ❖ One size fits all

❑ The AdaGrad Solution:

- ❖ Parameters with large/ frequent gradients → automatically get smaller learning rate
- ❖ Parameters with small/ rare gradients → automatically get larger learning rate

Things you've seen a lot of (frequent features)

— take small steps.

Things you've rarely seen — take big steps when you finally see them..

AdaGrad (Adaptive Gradient Algorithm)

Authors: John Duchi, Elad Hazan, and Yoram Singer

Year: 2011

Conference: COLT (Conference on Learning Theory) / JMLR

Previously, we performed an update for all parameters θ at once as every parameter θ_i used the same learning rate η . As AdaGrad uses a different learning rate for every parameter θ_i at every time step t , we first show AdaGrad's per-parameter update, which we then vectorize. For brevity, we set $g_{t,i}$ to be the gradient of the objective function w.r.t. to the parameter θ_i at time step t :

$$g_{t,i} = \nabla_{\theta_i} J(\theta_{t,i}) \quad (6)$$

The SGD update for every parameter θ_i at each time step t then becomes:

$$\theta_{t+1,i} = \theta_{t,i} - \eta \cdot g_{t,i} \quad (7)$$

In its update rule, AdaGrad modifies the general learning rate η at each time step t for every parameter θ_i based on the past gradients that have been computed for θ_i :

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii}} + \epsilon} \cdot g_{t,i} \quad (8)$$

$G_t \in \mathbb{R}^{d \times d}$ here is a diagonal matrix where each diagonal element i, i is the sum of the squares of the gradients w.r.t. θ_i up to time step $t^{[1]}$, while ϵ is a smoothing term that avoids division by zero (usually on the order of $1e-8$). Interestingly, without the square root operation, the algorithm performs much worse.

pra-sami

40

AdaGrad in Practice — Where It Shines

❑ AdaGrad significantly improves the robustness of standard SGD by:

- ❖ Removing the need to manually tune learning rates per parameter
- ❖ Adapting automatically to the frequency of each feature

original GloVe paper uses an AdaGrad-inspired adaptive learning rate schedule

❑ Being used for training large-scale neural nets

- ❖ At Google, to train GloVe word embeddings, as infrequent words require much larger updates than frequent ones..

❑ Previously, we performed an update for all Weights W at once as every parameter w_i used the same learning rate η (read α).

❑ As AdaGrad uses a different learning rate for every weight w_i at every time step t ,

❑ $\partial w_{t,i}$ to be the gradient of the objective function w.r.t. to the parameter w_i at time step t :

- ❖ $W_{t+1,i} = W_{t,i} - \frac{\eta}{\sqrt{G_{t,ii}} + \epsilon} \cdot \partial W_{t,i}$
- ❖ $G_{t,ii} = \sum_{\tau=1}^t (\partial w_{\tau,i})^2$
 - Where $G_{t,ii}$ is the sum of the element-wise multiplication (squaring) of the gradients until time-step t
- ❖ Out-of-box libraries available

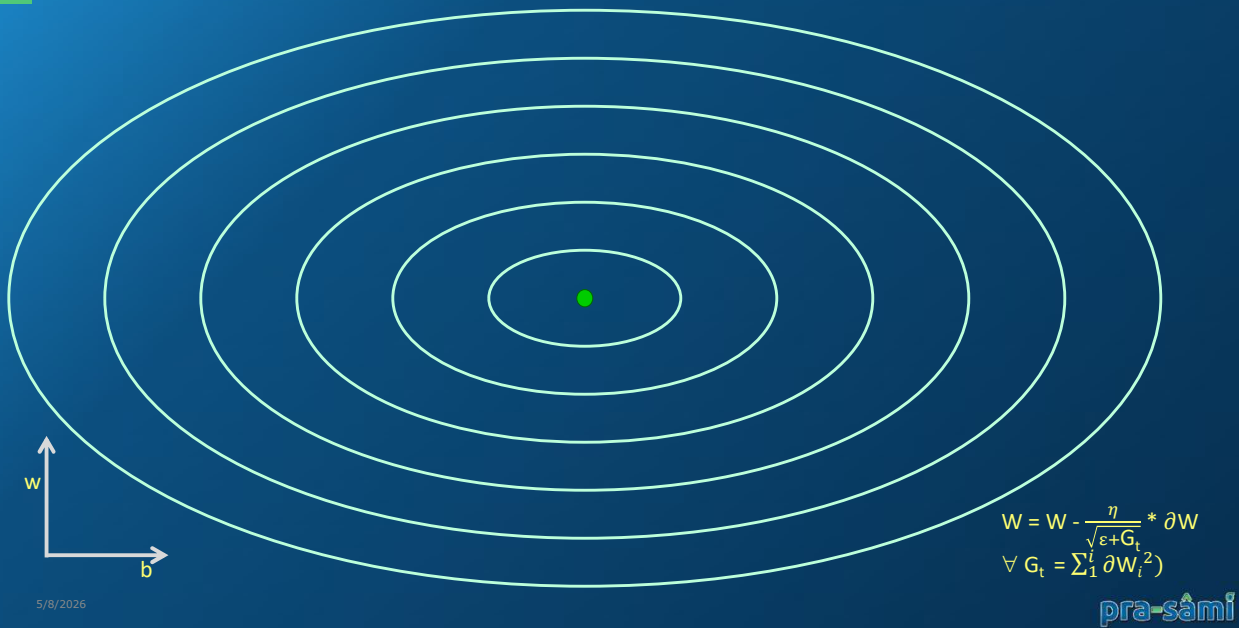
❑ Some libraries use diagonal matrix instead of using full matrix as it is computationally light weight

5/8/2026

pra-sami

41

AdaGrad



42

Adadelta — The Optimizer That Eliminates the Learning Rate

- ❑ Derived from AdaGrad in the Year 2012
 - ❖ Authors Matthew D. Zeiler
- ❑ To solve AdaGrad's aggressive, monotonically decreasing learning rate
- ❑ Instead of summing all past gradients, use Exponential Moving Average (EMA)
 - ❖ A window of recent history
- ❑ Removed alpha(α) from the equation

Step 1: Compute EMA of squared gradients (denominator)

$$G_t^2 = \rho \cdot G_{t-1}^2 + (1 - \rho) \cdot (\nabla J_t)^2$$

Step 2: Compute update (with units correction)

$$\Delta W_t = - \frac{\sqrt{E[\Delta W^2]_{t-1} + \epsilon}}{\sqrt{G_t^2 + \epsilon}} \cdot \nabla J_t$$

Step 3: Apply update

$$W_{t+1} = W_t + \Delta W_t$$

Step 4: Update EMA of squared updates (numerator)

$$E[\Delta W^2]_t = \rho \cdot E[\Delta W^2]_{t-1} + (1 - \rho) \cdot (\Delta W_t)^2$$

Innovation	What it does	Why it matters
1. Windowed gradients	EMA of squared gradients (like RMSProp)	Prevents LR from dying
2. Units correction	EMA of squared updates in numerator	Makes update unit-correct (Δw has same units as w)

5/8/2026

pra-sami

43

Benefits

No manual learning rate adjustment	The learning rate is completely eliminated — not just adaptive
Insensitive to hyperparameters	Default $p=0.95$ works for almost all problems
Separate dynamic LR per dimension	Inherited from AdaGrad — rare features get larger steps
Minimal computation	$O(n)$ — as cheap as standard SGD
Robust to large gradients & noise	EMA smooths out spikes
Robust to architecture choice	Works across CNNs, RNNs, Transformers
Works in local or distributed environments	No synchronization issues

5/8/2026

pra-sami

44

Rprop (Resilient Backpropagation) — Gradient Sign, Not Magnitude

- ❑ Each weight and bias has its own, variable, implied learning rate (α_i)

- ❑ Per-parameter learning rate

- ❖ Each weight w_i has its own step size α_i

- ❑ Ignores gradient magnitude

- ❖ Only uses sign of gradient ($\pm$)

In mini-batch, gradient sign can flip randomly between batches due to sampling noise. Rprop interprets this as "you passed a minimum" and **shrinks** α — even when you're far from the optimum.

- ❑ Adaptive step size

- ❖ α_i changes based on gradient sign consistency

- ❑ Why ignore gradient magnitude?

- ❖ In deep networks, gradient magnitudes vary wildly across layers.

- ❖ Rprop says: "I don't care how big the gradient is — I only care which direction it's pointing."

5/8/2026

pra-sami

45

RMSProp

RMSprop is an unpublished, adaptive learning rate method proposed by Geoff Hinton in Lecture 6e of his Coursera Class¹²

RMSprop and Adadelata have both been developed independently around the same time stemming from the need to resolve Adagrad's radically diminishing learning rates. RMSprop in fact is identical to the first update vector of Adadelata that we derived above:

$$\begin{aligned} E[g^2]_t &= 0.9E[g^2]_{t-1} + 0.1g_t^2 \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t \end{aligned} \quad (18)$$

Added exponential moving average on AdaGrad

RMSprop as well divides the learning rate by an exponentially decaying average of squared gradients. Hinton suggests γ to be set to 0.9, while a good default value for the learning rate η is 0.001.

Read β

Read α

5/8/2026

pra-sami

46

RMSProp — Normalizing Gradients by Their Recent Magnitude

Rprop works well for full-batch but fails for mini-batch. RMSProp adapts Rprop's "per-parameter step size" idea to work with mini-batch by using gradient magnitude normalization instead of sign-only.

□ Gradient for different weights are different

□ How RMSProp Works

- ❖ Keep a moving average of squared gradients for each weight
- ❖ Compute the Root Mean Square (RMS)
- ❖ Divide the current gradient by this RMS value
- ❖ Update weights using this normalized gradient

If recent gradients have been large, the current gradient is probably also large — so scale it down. If recent gradients have been small, scale it up.

5/8/2026

pra-sami

47

RMSProp

- Keep moving averages of squared gradients for each weight
 - ❖ $v_t^2 = \beta_2 \times v_{t-1}^2 + (1 - \beta_2) \times \left(\frac{\partial J}{\partial W}\right)^2$ (element wise square upto time step t)
- Then divide the gradient by square root the mean square above (Normalize it)

□ RMSProp updates :

$$\text{❖ } W = W - \frac{\eta}{\sqrt{v_t^2 + \epsilon}} * \frac{\partial J}{\partial W} \text{ (read } \alpha \text{ for } \eta)$$

Symbol	Name	Typical Value	What it does
η (or α)	Learning rate	0.001	Global step size scale
β_2	Decay rate	0.999	How much to remember past gradients
v_t^2	EMA of squared gradients	—	Running estimate of gradient magnitude
ϵ	Epsilon	1e-8	Prevents division by zero

pra-sami

48

Pros and Cons of RMSProp

Pros

- Works well with mini-batch
- No dying learning rate (unlike AdaGrad)
- Good for RNNs and non-stationary objectives
- Robust to gradient magnitude variations
- Computationally efficient ($O(n)$)

Cons

- Still requires learning rate tuning
- Can be unstable with some hyperparameters
- β_2 needs tuning (0.9 vs 0.999 vs 0.99)
- Outperformed by Adam in many tasks
- No momentum (can oscillate in ravines)

5/8/2026

pra-sami

49

When to Use RMSProp

- ❑ Recurrent Neural Networks (RNNs, LSTMs)
 - ❖ Excellent choice (historical favorite)
- ❑ Online learning / non-stationary problems
 - ❖ Very good
- ❑ When Adam overfits
 - ❖ Try RMSProp as alternative
- ❑ Avoid:
 - ❖ When you need momentum → Use Adam instead (RMSProp + Momentum)
 - ❖ Modern Transformers / LLMs → AdamW is preferred

RMSProp + Momentum = Adam

5/8/2026

pra-sâmi

50

Adam — The Best of Both Worlds (Momentum + RMSProp)

- ❑ Adam: A Method for Stochastic Optimization
- ❑ Adam was presented by Diederik Kingma (OpenAI) and Jimmy Ba (University of Toronto) in their 2015 ICLR paper (poster) titled “Adam: A Method for Stochastic Optimization”
 - ❖ <https://arxiv.org/abs/1412.6980>
- ❑ Citation: One of the most cited papers in deep learning (over 100,000+ citations)
- ❑ Its name Adam is derived from Adaptive Moment Estimation
- ❑ It combines momentum and RMSProp to adapt learning rates
- ❑ Adam with L2 regularization generalizes worse than AdamW (but still trains)
- ❑ AdamW decouples weight decay from adaptive updates to fix generalization

5/8/2026

pra-sâmi

51

Adaptive Moment Estimation (Adam)

- Adam = Momentum + RMSProp
- First moment (m)
 - ❖ EMA of gradients → smooth direction, build speed
- Second moment (v)
 - ❖ EMA of squared gradients → normalize step size per dimension

Why choose between momentum and RMSProp? Use both!

5/8/2026

pra-sami

52

Adaptive Moment Estimation (Adam)

- Parameters:
- $\alpha = 0.001$,
- $\beta_1 = 0.9$,
- $\beta_2 = 0.999$ and
- $\epsilon = 10^{-8}$

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1]$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

5/8/2026

pra-sami

53

Why Adam Works So Well

- ❑ Adaptive learning rates
 - ❖ Each parameter has its own effective LR (from RMSProp)
 - ❑ Momentum smoothing
 - ❖ Faster convergence, escapes local minima (from Momentum)
 - ❑ Bias correction
 - ❖ Fixes initialization bias for early training steps
 - ❑ Robust to hyperparameters
 - ❖ Works well with default values out-of-the-box
- ❑ Problem with original Adam:
 - ❖ Weight decay (L2 regularization) is implemented incorrectly. It interacts with the adaptive learning rates in unintended ways.
 - ❑ AdamW (2017, Loshchilov & Hutter):
 - ❖ Decouples weight decay from the adaptive updates. Use this instead of Adam.

5/8/2026

pra-sâmi

54

Pros and Cons of Adam

Pros	Cons
❑ Works well out-of-the-box	❑ Can generalize worse than SGD on some tasks
❑ Computationally efficient ($O(n)$)	❑ Requires more memory (stores m and v)
❑ Handles sparse gradients well	❑ Can be unstable with some hyperparameters
❑ Suitable for large-scale problems	❑ Weight decay not correctly implemented (AdamW fixes this)
❑ Robust to noisy gradients	❑ May converge to sharp minima
❑ Little hyperparameter tuning needed	❑ Sometimes overfits compared to SGD

5/8/2026

pra-sâmi

55

When to Use Adam

New problem / unknown terrain	Start with Adam (default choice)
NLP / Transformers / LLMs	⚠ Use AdamW (Adam with correct weight decay)
Computer Vision (ImageNet)	⚠ Try SGD + Momentum (often generalizes better)
GANs	Adam is standard
Reinforcement Learning	Good choice
When you need peak test accuracy	⚠ Tune both Adam and SGD, compare

□ Adam incorporates three key ideas

□ Stochastic mini-batch sampling (for efficiency and noise)

Not a part of original paper

□ Momentum (first moment) (to smooth gradients and build speed)

□ RMSProp (second moment) (to adapt learning rates per parameter)

5/8/2026

pra-sami

56

AdamW

Algorithm 2 Adam with L_2 regularization and Adam with decoupled weight decay (AdamW)

```

1: given  $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$ 
2: initialize time step  $t \leftarrow 0$ , parameter vector  $\theta_{t=0} \in \mathbb{R}^n$ , first moment vector  $m_{t=0} \leftarrow \mathbf{0}$ , second moment vector  $v_{t=0} \leftarrow \mathbf{0}$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$  ▷ select batch and return the corresponding gradient
6:    $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ 
7:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$  ▷ here and below all operations are element-wise
8:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
9:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  ▷  $\beta_1$  is taken to the power of  $t$ 
10:   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  ▷  $\beta_2$  is taken to the power of  $t$ 
11:   $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$  ▷ can be fixed, decay, or also be used for warm restarts
12:   $\theta_t \leftarrow \theta_{t-1} - \eta_t \left( \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1} \right)$ 
13: until stopping criterion is met
14: return optimized parameters  $\theta_t$ 

```

5/8/2026

pra-sami

57

Which one to use???

- ❑ For sparse data, use one of the adaptive learning-rate methods
- ❑ RMSprop, Adadelata, and Adam are very similar algorithms.
 - ❖ Pick any and then try others
- ❑ Adam slightly outperform RMSprop towards the end of optimization as gradients become sparser
- ❑ Adam or AdamW may appear to be the best overall choice
- ❑ Few modern papers use SGD without momentum
 - ❖ SGD usually achieves a minimum, but it might take significantly longer than with some of the optimizers,
- ❑ For fast convergence and train a deep or complex neural network, you should choose one of the adaptive learning rate methods

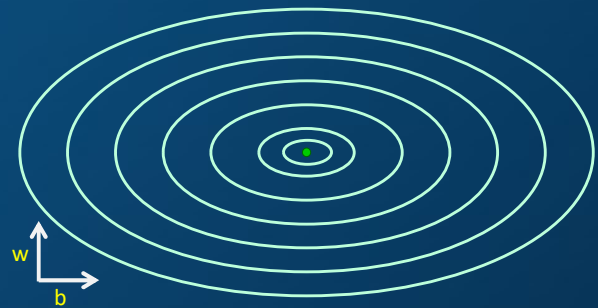
5/8/2026

pra-sâmi

58

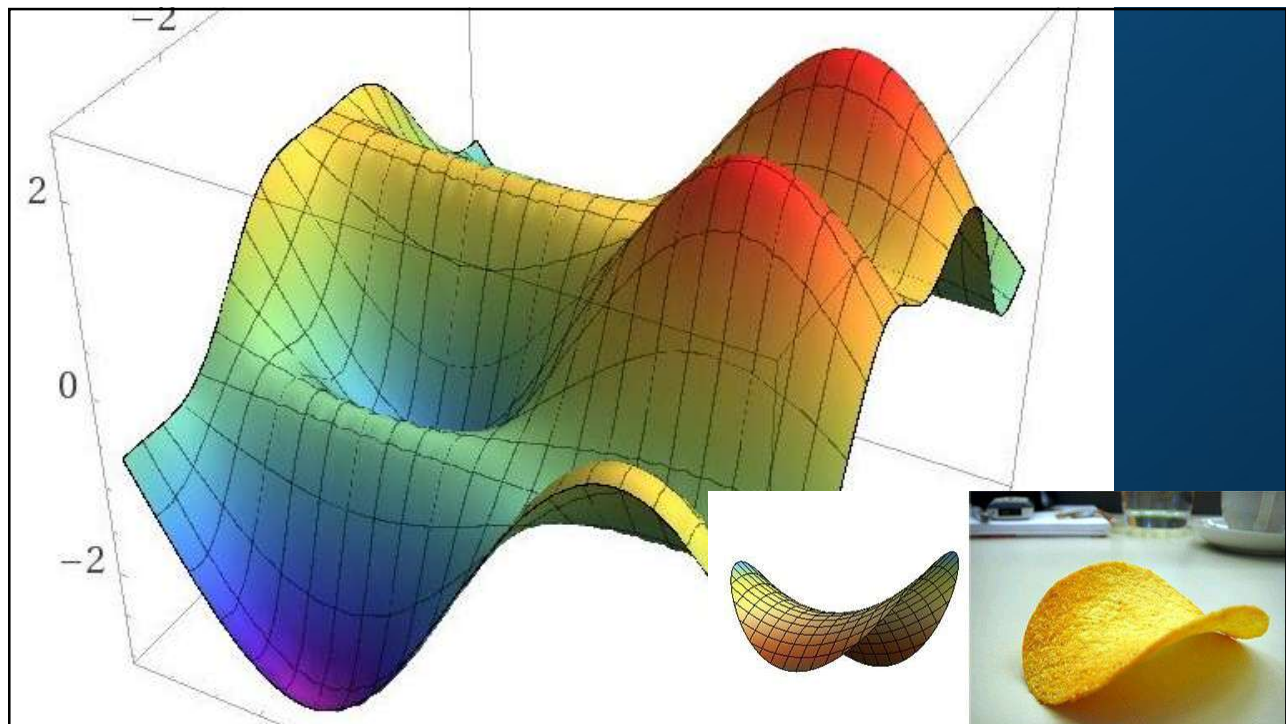
Learning Rate Decay

- ❑ Idea is to take advantage of large steps in the beginning and then reduce your step size
 - ❖ With some threshold at a minimum value
- ❑ If you continue with the same step size,
 - ❖ You might keep hopping around
- ❑ A few recommended techniques:
 - ❖ $\alpha = \frac{1}{1 - \text{decay rate} * \text{epoch number}} * \alpha_0$
 - ❖ Or $\alpha = (0.98)^{\text{epochnum}} * \alpha_0$
 - ❖ Or may be stepped; reduce by 0.1 * α_{t-1} after 100 epochs



5/8/2026

pra-sâmi



60

Recent Development

5/8/2026

pra-sâmí

61

Lion (EvoLved Sign Momentum) — Memory-Efficient Vision Training

- ❑ Lion uses only the SIGN of the momentum
 - ❖ no second moment (no v_t)
- ❑ Updates :
 - ❖ Momentum update: $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla J(W_{t-1})$
 - ❖ Weight update (Lion): $W_t = W_{t-1} - \eta \cdot \text{sign}(m_t) - \lambda \cdot W_{t-1}$
- ❑ Lion ignores how large the gradient is
 - ❖ only cares which direction it's pointing
- ❑ Use when memory is limited
 - ❖ e.g. Edge Devices

Title	Symbolic Discovery of Optimization Algorithms
Authors	Xiangning Chen, Chen Liang, Da Huang, et al. (Google Research)
Year	2023
Venue	NeurIPS 2023
Key insight	Evolutionary search discovered a simpler, better optimizer

5/8/2026

pra-sâmi

62

Lion Summary

- ❑ Sign-based optimizer with 50% memory — best for vision
- ❑ Key Equations:
 - ❖ $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla J$
 - ❖ $W_t = W_{t-1} - \eta \cdot \text{sign}(m_t) - \lambda \cdot W_{t-1}$
- ❑ Memory: 1× model size (half of AdamW)
- ❑ Best for: Vision, memory-constrained training
- ❑ Default: $\alpha=1e-4$, $\beta_1=0.9$, $\lambda=0.01$

5/8/2026

pra-sâmi

63

Sophia (Second-order Clipped Stochastic Optimization)

□ Sophia also uses curvature (second-order) for smarter steps

- ❖ Adam uses only gradient (first-order).
- ❖ Curvature tells you how confident you should be in the gradient direction.

□ Updates:

- ❖ First moment (like Adam): $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla J(W_{t-1})$
- ❖ Hessian diagonal estimate (curvature): $h_t \approx \text{diag}(\nabla^2 J(W_t))$
- ❖ Hutchinson trick (efficient estimation): $h_t = \frac{1}{k} \sum_{i=1}^k \text{diag}(\nabla^2 J(W_t) \cdot z_i) \odot z_i$
- ❖ Sophia update (clipped): $W_t = W_{t-1} - \eta \cdot \text{clip}\left(\frac{m_t}{\max(\rho \cdot h_t, \epsilon)}, 1\right)$

Title	Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training
Authors	Hong Liu, Zhiyuan Li, David Leo Wright Hall, et al. (Stanford, UT Austin)
Year	2023
Venue	arXiv (ICLR 2024 submission)
Key insight	Use diagonal Hessian (curvature) for faster convergence

LLM pre-training (GPT, LLaMA) : Best choice (2× faster)

5/8/2026

pra-sami

64

Sophia Summary

□ Second-order optimizer that trains LLMs 2× faster than AdamW

□ Key Innovation:

- ❖ Uses diagonal Hessian (curvature) to clip updates
- ❖ $m_t = \text{EMA}(\text{gradient})$
- ❖ $h_t = \text{diagonal Hessian estimate (via Hutchinson trick)}$
- ❖ $W \leftarrow W - \eta \cdot \text{clip}(m_t / (\rho \cdot h_t), 1)$

□ Convergence: 2× faster than AdamW on GPT pre-training

□ Best for: LLMs, large-scale pre-training

□ Default: $\alpha=2e-4$, $\rho=0.04$, update Hessian every 10 steps

5/8/2026

pra-sami

65

LAMB (Layer-wise Adaptive Moments for Batch training)

- ❑ LAMB enables batch sizes up to 65,536
 - ❖ a $1000 \times$ increase!
- ❑ Updates:
 - ❖ Standard Adam update (per layer): $\Delta W_t^{\text{Adam}} = -\eta \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$
 - ❖ LAMB's trust ratio (per layer): $r_t = \frac{\|W_t\|}{\|\Delta W_t^{\text{Adam}}\| + \epsilon}$
 - ❖ LAMB update: $W_{t+1} = W_t - \eta \cdot r_t \cdot \frac{m_t}{\sqrt{v_t} + \epsilon} - \lambda \cdot W_t$
- ❑ Distributed training with many GPUs (8+)
- ❑ Pre-training BERT, GPT, ViT at scale – Industry standard
- ❑ Batch size > 4K

Title	Large Batch Optimization for Deep Learning: Training BERT in 76 Minutes
Authors	Yang You, Jing Li, Sashank Reddi, et al. (Google, UC Berkeley)
Year	2019
Venue	ICLR 2020
Key insight	Layer-wise adaptation for extreme batch sizes

5/8/2026

pra-sâmi

66

LAMB Summary

- ❑ Scale batch size to 64K+ for distributed training
- ❑ Key Innovation (Trust Ratio):
 - ❖ $r = \|W\| / (\|\Delta \text{Adam}\| + \epsilon)$
 - ❖ $W \leftarrow W - \eta \cdot r \cdot (m/v) - \lambda \cdot W$
- ❑ Achievement: BERT in 76 minutes (vs 100 hours)
- ❑ Best for: Large-batch distributed training
- ❑ Default: Batch size 32K, $\alpha=0.001$

5/8/2026

pra-sâmi

67

Adafactor — Factorized 2nd Moment for Memory-Constrained Training

□ Adafactor reduces memory from $O(n^2)$ to $O(n + m)$

□ Updates:

- ❖ Adam's second moment: $V_t = \beta_2 \cdot V_{t-1} + (1 - \beta_2) \cdot g_t^2$
 - ($m \times n$ matrix)
- ❖ Adafactor's factorization trick: $V_t \approx R_t \cdot C_t^T$
- ❖ Update rule: $\hat{V}_t = \frac{R_t \cdot C_t^T}{\sum C_t}$
- ❖ Weight update: $W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{V}_t + \epsilon}} \cdot g_t$
- ❖ Where:
 - R_t = running average of row sums (size m)
 - C_t = running average of column sums (size n)

Title	Adafactor: Adaptive Learning Rates with Sublinear Memory Cost
Authors	Noam Shazeer, Mitchell Stern (Google Brain)
Year	2018
Venue	ICML 2018
Key insight	Factorize second-moment matrices to save memory

□ Adafactor reduces optimizer memory by 100-1000× for large embedding layers.

5/8/2026

pra-sâmi

68

Adafactor - Summary

□ Factorized second moment — 200× memory savings for large layers

□ Key Innovation:

- ❖ $V \approx R \cdot C^T$ (rank-1 factorization)
- ❖ Memory: $O(m+n)$ instead of $O(m \times n)$

□ Memory savings: 100-1000× for embedding layers

□ Best for: Memory-constrained training, large embeddings

□ Default: Use `relative_step=True` (adaptive LR)

5/8/2026

pra-sâmi

69

Reflect...

- ❑ In Gradient Descent, what does the "gradient" represent?
 - ❖ A. The direction of steepest ascent of the function.
 - ❖ B. The rate of change of the function at a point.
 - ❖ C. The distance between data points.
 - ❖ D. The probability of occurrence of a data point.
- ❑ Answer: B. The rate of change of the function at a point.
- ❑ What is the role of the learning rate in Gradient Descent?
 - ❖ A. It determines the number of iterations.
 - ❖ B. It specifies the initial position of the algorithm.
 - ❖ C. It controls the size of the steps taken during optimization.
 - ❖ D. It sets the threshold for convergence.
- ❑ Answer: C. It controls the size of the steps taken during optimization.

5/8/2026

pra-sami

70

Reflect...

- ❑ Which variant of Gradient Descent updates parameters after computing the gradient on the ENTIRE training dataset?
 - ❖ A. Stochastic Gradient Descent (SGD)
 - ❖ B. Mini-batch Gradient Descent
 - ❖ C. Batch Gradient Descent
 - ❖ D. Momentum-based Gradient Descent
- ❑ Answer: C. Batch Gradient Descent
- ❑ Which statement best describes the trade-off addressed by Mini-batch Gradient Descent?
 - ❖ A. Batch GD is faster than SGD in terms of wall-clock time
 - ❖ B. SGD guarantees convergence to the global minimum
 - ❖ C. Mini-batch GD balances computational efficiency (batch) with stochastic noise (SGD)
 - ❖ D. Momentum-based GD is less prone to local minima than mini-batch
- ❑ Answer: C

5/8/2026

pra-sami

71

Reflect...

- ❑ What is the primary consequence of setting the learning rate TOO HIGH in Gradient Descent?
 - ❖ A. Slow convergence and potentially getting stuck in plateaus
 - ❖ B. Overshooting the minimum and possibly diverging
 - ❖ C. Guaranteed convergence to the global minimum
 - ❖ D. Reduced computational complexity per iteration
- ❑ Answer: B. Overshooting the minimum and possibly diverging
- ❑ Which statement about Gradient Descent convergence is TRUE for non-convex loss functions (like deep neural networks)?
 - ❖ A. Gradient Descent always converges to the global minimum
 - ❖ B. Gradient Descent may converge to a local minimum or saddle point depending on initialization and learning rate
 - ❖ C. Gradient Descent never converges for non-convex functions
 - ❖ D. Gradient Descent converges faster with a smaller number of iterations
- ❑ Answer: B. Gradient Descent may converge to a local minimum or saddle point depending on initialization and learning rate

5/8/2026

pra-sâmi

72

Reflect...

- ❑ Which technique directly addresses the problem of oscillations around steep ravines in the loss landscape?
 - ❖ A. Decreasing the learning rate over time (learning rate decay)
 - ❖ B. Increasing the learning rate over time
 - ❖ C. Using a larger batch size
 - ❖ D. Adding momentum to the updates
- ❑ Answer: D. Adding momentum to the updates
- ❑ Which of the following is NOT an optimization algorithm for training neural networks?
 - ❖ A. AdaGrad
 - ❖ B. K-means
 - ❖ C. RMSProp
 - ❖ D. Adam
- ❑ Answer: B. K-means

5/8/2026

pra-sâmi

73

Reflect...

- ❑ In the Adam optimizer, what do the hyperparameters β_1 and β_2 control?
 - ❖ A. The learning rate warmup schedule and weight decay factor
 - ❖ B. The exponential decay rates for first moment (mean) and second moment (uncentered variance) of gradients
 - ❖ C. The mini-batch size and number of training epochs
 - ❖ D. The initialization scale for weights and biases
- ❑ Answer: B. The exponential decay rates for first moment (mean) and second moment (uncentered variance) of gradients
- ❑ Which statement correctly describes a KEY mathematical difference between Momentum and RMSProp?
 - ❖ A. Momentum uses exponential moving averages; RMSProp uses a fixed window
 - ❖ B. Momentum tracks the first moment (mean of gradients); RMSProp tracks the second moment (mean of squared gradients)
 - ❖ C. Momentum works only for convex functions; RMSProp works for all functions
 - ❖ D. RMSProp requires tuning of learning rate; Momentum does not
- ❑ Answer: B. Momentum tracks the first moment (mean of gradients); RMSProp tracks the second moment (mean of squared gradients)

5/8/2026

pra-sâmi

74

Reflect...

- ❑ What is the fundamental problem with standard Adam that AdamW corrects?
 - ❖ A. Adam's learning rate decays to zero over time (like AdaGrad)
 - ❖ B. Adam's implementation of weight decay (L2 regularization) is coupled with adaptive learning rates, leading to poor generalization
 - ❖ C. Adam cannot handle sparse gradients effectively
 - ❖ D. Adam does not work with mini-batch training
- ❑ Answer: B. Adam's implementation of weight decay (L2 regularization) is coupled with adaptive learning rates, leading to poor generalization
 - ❖ Explanation: In Adam, L2 regularization (adding λW to gradient) gets divided by $\sqrt{v}$, causing uneven regularization across parameters. AdamW decouples weight decay (applying $\eta \cdot \lambda \cdot W$ after the adaptive update), fixing generalization.
- ❑ For which task is SGD with Momentum (without adaptive learning rates) often preferred over AdamW?
 - ❖ A. Large Language Model training (GPT, LLaMA)
 - ❖ B. Image classification on ImageNet with CNNs (ResNet, EfficientNet)
 - ❖ C. Sparse data problems (recommender systems, word embeddings)
 - ❖ D. Training Generative Adversarial Networks (GANs)
- ❑ Answer: B. Image classification on ImageNet with CNNs (ResNet, EfficientNet)
 - ❖ Explanation: For CNNs on dense vision tasks (ImageNet), SGD + Momentum often achieves better test accuracy/generalization than AdamW, though it requires careful learning rate scheduling and more epochs. LLMs (A) use AdamW. Sparse data (C) favors adaptive methods. GANs (D) typically use Adam.

5/8/2026

pra-sâmi

75

Reflect...

❑ Which advantage does mini-batch gradient descent have over full batch gradient descent?

- ❖ A. It eliminates the need for backpropagation
- ❖ B. It allows faster and more frequent updates
- ❖ C. It always converges to the global minimum
- ❖ D. It doesn't require a loss function

❑ Answer: B. It allows faster and more frequent updates

❑ Which of the following is not a common optimization algorithm used in deep learning?

- ❖ A. Adam
- ❖ B. Adagrad
- ❖ C. Newton-Raphson
- ❖ D. RMSProp

❑ Answer: C. Newton-Raphson

5/8/2026

pra-sâmi

76

Reflect...

❑ Which optimizer eliminates the learning rate hyperparameter entirely?

- ❖ A. Adam
- ❖ B. RMSProp
- ❖ C. Adadelta
- ❖ D. AdaGrad

❑ Answer: C. Adadelta

- ❖ Explanation: Adadelta is unique in that it eliminates the learning rate parameter by using the RMS of past updates in the numerator (units correction). All others (Adam, RMSProp, AdaGrad, SGD) require a learning rate to be set. Adadelta uses ρ (decay rate) and ϵ only.

❑ Why does Rprop (Resilient Backpropagation) fail in modern deep learning with mini-batches?

- ❖ A. It requires too much memory for large models
- ❖ B. It uses only the sign of the gradient, which fluctuates randomly across mini-batches, causing unstable step sizes
- ❖ C. It does not support per-parameter learning rates
- ❖ D. It cannot be parallelized across GPUs

❑ Answer: B. It uses only the sign of the gradient, which fluctuates randomly across mini-batches, causing unstable step sizes

- ❖ Explanation: Rprop works well with full-batch (stable gradient sign) but fails with mini-batches because gradient sign flips randomly due to sampling noise, causing the delta values to shrink incorrectly. It is not available in PyTorch/TensorFlow.

5/8/2026

pra-sâmi

77

Reflect...

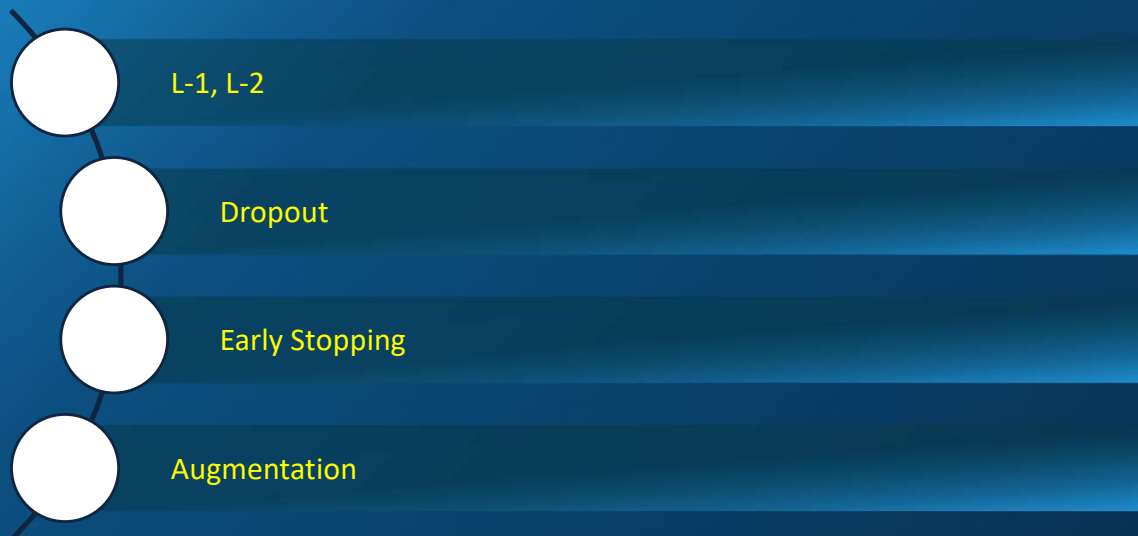
- ❑ In Nesterov Accelerated Gradient (NAG), where is the gradient computed, and why?
 - ❖ A. At the current position — to follow the steepest descent
 - ❖ B. At the lookahead position ($W - \alpha \cdot V_t$) — to anticipate the momentum's effect and correct overshoot
 - ❖ C. At the previous position — to maintain stability
 - ❖ D. At a random position — to add stochastic noise
- ❑ Answer: B. At the lookahead position ($W - \alpha \cdot V_t$) — to anticipate the momentum's effect and correct overshoot
 - ❖ Explanation: NAG computes gradient at the lookahead position (where current momentum would take you), then uses this gradient to correct the velocity before updating. This "look before you leap" approach reduces overshooting and improves convergence, especially near minima.
- ❑ In which situation would Nesterov Accelerated Gradient (NAG) perform better than traditional momentum-based gradient descent?
 - ❖ A. When the learning rate is fixed
 - ❖ B. When the gradient updates are sparse
 - ❖ C. When the learning rate is adaptive
 - ❖ D. When faster convergence is needed, especially in high-dimensional spaces
- ❑ Answer: D. When faster convergence is needed, especially in high-dimensional spaces

5/8/2026

pra-sâmi

78

Next Session



5/8/2026

pra-sâmi

79

Over to Jupyter Notebook

5/8/2026

pra-sâmi

80

THANK YOU

5/8/2026

pra-sâmi

ADDITIONAL MATERIAL



What's Next

82

Augmented Random Search

Horia Mania Aurelia Guy Benjamin Recht
Department of Electrical Engineering and Computer Science
University of California, Berkeley
March 20, 2018

5/8/2026



83

Augmented Random Search

- ❑ It's a Shallow Learning Algorithm
- ❑ Using Random Noise
- ❑ Exploits Generic Evolution Theory

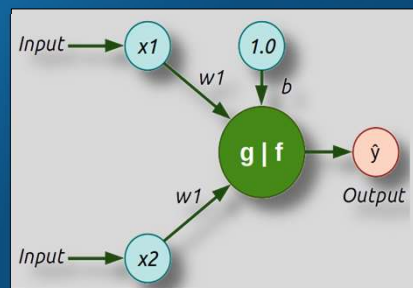
5/8/2026

pra-sâmi

84

Perceptron

- ❑ Welcome back our perceptron

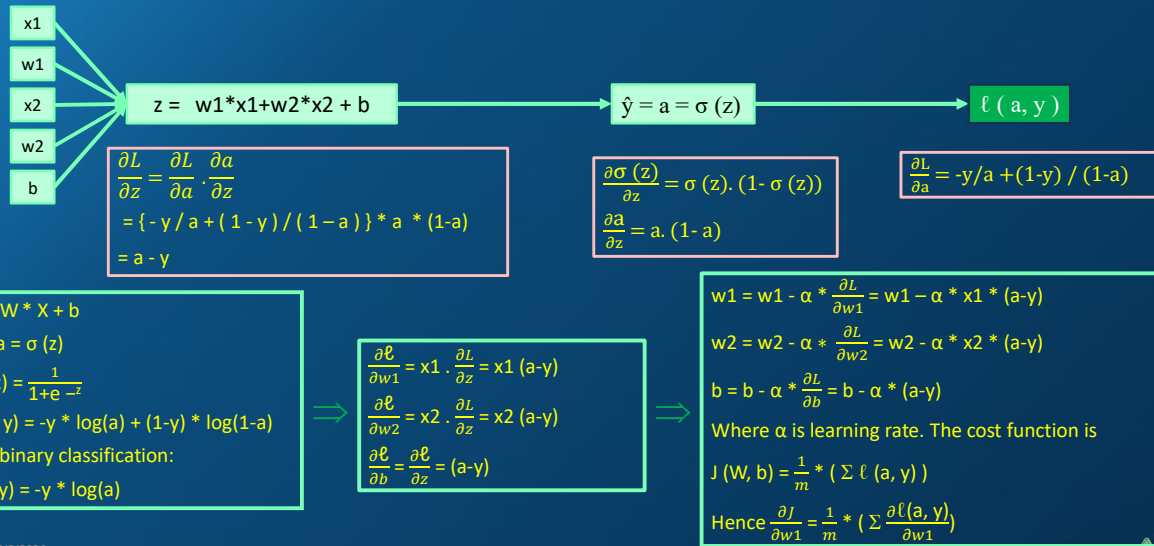


5/8/2026

pra-sâmi

85

Perceptron

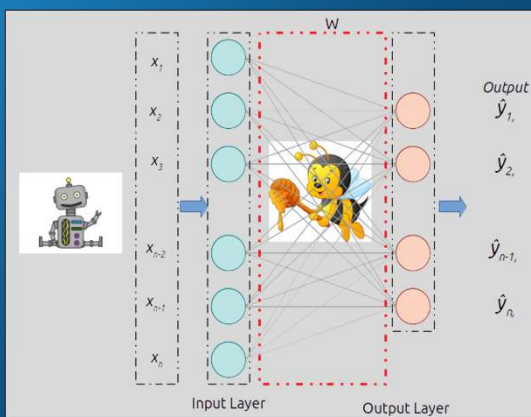


5/8/2026

pra-sami

86

Perceptron



Shape of Weights W (inputs, outputs)

- Earlier we were using very elaborate calculations to estimate direction of gradient descent.....
- Why???
- Can't we just calculate in both directions and see what's better?
- Not a bad suggestion... lets work out a procedure....
- Add random noise to weights W
- Run a trial run
- Move in the direction of improvement

5/8/2026

pra-sami

87

Method of Finite Difference

- ❑ Generate small random numbers (δW)
- ❑ Shape of δW will be same as that of W
- ❑ Create W_1 and W_2 as follows
 - ✧ $W_1 = W + \delta W$
 - ✧ $W_2 = W - \delta W$
- ❑ Test both version and record Loss from each G_p, G_n
- ❑ Update Weights as follows
- ❑ $W = W + \alpha (G_p - G_n) \cdot \delta W$
- ❑ Keep repeating till it converges to a Minima (We are minimizing the cost!)
- ❑ Can easily be implemented for gain in a similar fashion

5/8/2026

pra-sâmi

88

Learning Rate Decay

- ❑ Slowly reduce learning rate.
- ❑ As mentioned before mini-batch gradient descent won't reach the optimum point (converge). But by making the learning rate decay with iterations it will be much closer to it because the steps (and possible oscillations) near the optimum are smaller.
- ❑ One equations is
 - ✧ $\text{learning_rate} = (1 / (1 + \text{decay_rate} * \text{epoch_num})) * \text{learning_rate_0}$
 - ✧ epoch_num is over all data (not a single mini-batch)
- ❑ Other learning rate decay methods (continuous):
 - ✧ $\text{learning_rate} = (0.95^{\text{epoch_num}}) * \text{learning_rate_0}$
 - ✧ $\text{learning_rate} = (k / \text{sqrt}(\text{epoch_num})) * \text{learning_rate_0}$
- ❑ Some people perform learning rate decay discretely - repeatedly decrease after some number of epochs
- ❑ Some people are making changes to the learning rate manually
 - ✧ 'decay_rate' is another hyperparameter
- ❑ Learning rate decay has less priority... last thing to tune in your network

5/8/2026

pra-sâmi